

LPOO · IFSul · Ciência da Computação

Sistema de Gestão de Sócio-Torcedores

Documentação do projeto

1. Português · 2. English · 3. Leitura Fácil

Allan Theodoro · Bernardo Zavistanovicz Deters

Olavo Appel Neto · Thomas Cansian dos Santos

Documentação do Sistema de Gestão de Sócio-Torcedores

Projeto da disciplina de Linguagem de Programação Orientada a Objetos (LPOO) — Ciência da Computação, IFSul.

Equipe: Allan Theodoro, Bernardo Zavistanovicz Deters, Olavo Appel Neto e Thomas Cansian dos Santos.

Versões deste documento: Português (este arquivo) · English · Leitura Fácil

1. Introdução

Este documento descreve o Sistema de Gestão de Sócio-Torcedores. Ele explica o que o sistema faz, como o código está organizado e por que a equipe tomou cada decisão de projeto.

O texto usa frases curtas, voz ativa e um termo fixo para cada conceito. A seção 12 traz o glossário.

O documento descreve o código **como ele está hoje**. O que ainda não funciona aparece de forma clara na seção 3.3 e na seção 11.

Os diagramas UML ficam na seção 8. Os arquivos-fonte dos diagramas ficam na pasta `docs/diagramas/`.

2. Objetivo do sistema

O sistema ajuda um clube de futebol a gerir seus sócio-torcedores. A proposta do projeto prevê quatro áreas:

1. **Gestão de associados:** cadastro de novos associados e troca de plano.
2. **Categorias e descontos:** classes de sócios, com preços e descontos para cada classe.
3. **Controle de ingressos:** valores por campeonato e controle da venda dos ingressos dos jogos.
4. **Programa de benefícios:** controle de brindes e de experiências exclusivas.

Nesta entrega, o sistema cobre o cadastro de usuários, o login, o calendário de jogos e a lista de lotes de ingresso de cada jogo. As demais áreas ainda estão em desenvolvimento (seção 3.3).

Nesta disciplina, o projeto também serve para mostrar quatro competências: documentação, padrões de projeto, interface gráfica e padrões de arquitetura.

3. Descrição do sistema

3.1 Tecnologias

Item	Tecnologia
Linguagem	Java 26
Framework principal	Spring Boot 4.1.1
Gerenciador de build	Maven (com <code>mvnw</code>)
Camada web	Spring MVC e Thymeleaf
Persistência	Spring Data JPA (Hibernate)
Segurança	Spring Security, com senhas em BCrypt
Validação	Bean Validation
Redução de código repetido	Lombok
Banco de dados	PostgreSQL
Interface	HTML, CSS e JavaScript

3.2 Funcionalidades prontas

- **Cadastro de usuário.** O visitante informa usuário, e-mail, senha e CPF. O sistema cria um `Usuario` com um `Associado` da categoria `NAO_ASSOCIADO`.
- **Login e logout.** O sistema usa formulário de login. Há dois papéis: `USER` e `ADMIN`.

- **Calendário de jogos.** A tela mostra os jogos do ano, o destaque do próximo jogo e um filtro por mês.
- **Lotes de ingresso.** A tela mostra os lotes de um jogo. Cada lote aparece como válido ou inválido, com o motivo.
- **Telas Perfil e Admin.** As duas telas mostram o nome do usuário. A tela Admin só abre para o papel `ADMIN`.
- **Dados iniciais.** A classe `DataSeeder` cria o usuário `admin`, seis times, dois estádios, seis jogos e quatro lotes para o próximo jogo.

3.3 Funcionalidades ainda não prontas

Item	Situação no código
Compra de ingresso	O botão Comprar existe. O endereço <code>/ingressos/comprar</code> não tem controller. O método <code>IngressoService.comprarIngresso()</code> está vazio.
Tornar-se sócio	<code>SocioTorcedorController</code> está comentado. <code>SocioTorcedorService.tornarseSocio()</code> existe, mas nenhuma classe o chama.
Desconto por categoria	O preço com desconto é igual ao preço base (em <code>DTOfactory</code>).
Programa de benefícios	A entidade <code>BeneficioSocio</code> existe, mas nenhuma classe a usa.
Botão Saiba Mais	O link aponta para <code>#</code> .

3.4 Como executar

1. Instale o JDK 26 e o PostgreSQL.
2. Crie um banco de dados vazio.
3. Defina as variáveis de ambiente `DB_NAME`, `DB_USER` e `DB_PASSWORD`.
4. Na pasta `sociotorcedor/`, execute `./mvnw spring-boot:run`.
5. Abra `http://localhost:8080`.

O banco usa o endereço `localhost:5432`. Na primeira execução, o `DataSeeder` cria os dados iniciais. O usuário inicial de demonstração é `admin`, com a senha `admin`.

3.5 Armazenamento de dados

O sistema guarda os dados no PostgreSQL. A propriedade `spring.jpa.hibernate.ddl-auto=update` faz o Hibernate criar e atualizar as tabelas. O sistema não usa arquivos de migração.

4. Arquitetura

O sistema usa **arquitetura em camadas** e, na parte web, o padrão **MVC** do Spring. A seção 6 explica as duas escolhas. Esta seção mostra a organização.

4.1 Camadas

Camada	Pacotes	Responsabilidade
Apresentação	<code>templates/</code> , <code>static/</code> , <code>api/controller</code>	Recebe o pedido do navegador. Chama o serviço. Escolhe a tela.
Serviço	<code>api/service</code> , <code>core/validation</code> , <code>core/factory</code>	Aplica as regras de negócio. Valida. Cria objetos.
Persistência	<code>api/repository</code>	Lê e grava os dados no banco.
Domínio e transferência de dados	<code>core/model</code> , <code>core/dto</code> , <code>core/form</code>	Define as entidades e os objetos que viajam entre as camadas.
Transversal	<code>security</code> , <code>config</code> , <code>migration</code>	Configura a segurança, a cadeia de validação e os dados iniciais.

4.2 Como as camadas se comunicam

- O navegador fala só com os controllers.
- Os controllers falam só com os serviços. Nenhum controller acessa o banco.
- Os serviços usam os repositórios, as fábricas e a cadeia de validação.
- Os repositórios falam com o PostgreSQL.
- Todas as camadas usam as classes do domínio.

A Figura 3 (seção 8) mostra essa organização.

4.3 Exemplo de um pedido

O usuário abre `/ingressos?jogoId=1`. O `LoteIngressoController` cria um `ContextoCompra` e chama o `LoteIngressoService`. O serviço busca o jogo no `JogoRepository`. Depois, o serviço valida cada lote com a cadeia de validação. O `DTOfactory` cria um `LoteIngressoDTO` para cada lote. O controller envia a lista para a tela `ingressos/lista`. A Figura 4 mostra esse fluxo.

5. Padrões de projeto utilizados

A equipe usa cinco padrões de projeto. Cada padrão tem uma classe ou um grupo de classes que o implementa.

5.1 Chain of Responsibility (Cadeia de Responsabilidade)

- **Problema.** As regras para vender um lote vão crescer. Colocar todas em um único método cria um `if` grande e difícil de mudar.
- **Onde aparece.** `LoteValidationHandle`, `IntervaloLoteValidator`, `LoteValidationConfig` e `LoteIngressoService`.
- **Como funciona.** Cada regra é uma classe. Cada classe recebe o lote, valida e passa o lote para a próxima classe da cadeia. A cadeia para na primeira regra que falha. O `LoteValidationConfig` monta a cadeia. O `LoteIngressoService` só chama a primeira classe.
- **Situação atual.** A cadeia tem **uma regra**: `IntervaloLoteValidator`. Ela verifica se o horário do pedido está dentro do período de venda do lote. Para criar uma nova regra, a equipe escreve uma nova classe e liga a classe com `setProximo()`. O modelo já tem dados para novas regras, como `categoriaMinima` e `ingressoDisponiveis`. Essas regras ainda não existem.
- **Por que serve.** O `LoteIngressoService` não muda quando uma regra nova entra. Isso segue o princípio aberto/fechado.

5.2 Template Method (Método Modelo)

- **Problema.** Todas as regras da cadeia precisam seguir o mesmo fluxo: validar, parar se falhou, chamar a próxima regra. Cada regra não pode refazer esse fluxo.
- **Onde aparece.** `LoteValidationHandle`.
- **Como funciona.** O método `handle()` é `final`. Ele define a ordem dos passos. O método `validar()` é abstrato. Cada subclasse implementa só o `validar()`.
- **Por que serve.** O fluxo da cadeia fica em um único lugar. Nenhuma subclasse consegue mudá-lo.

5.3 Factory (Fábrica simples)

- **Problema.** Criar alguns objetos exige vários passos. Espalhar esses passos pelo código gera repetição e erro.
- **Onde aparece.**
 - `UsuarioFactory.createNewUsuario()` monta um `Usuario` com um `Associado`, define a categoria `NAO_ASSOCIADO` e o papel `USER`, e criptografa a senha.
 - `DTOfactory` converte entidades em DTOs (`createJogoDTO` e `createLoteIngressoDTO`).
- **Por que serve.** O `CadastroService` e o `JogoService` não conhecem os detalhes da criação. Se a criação mudar, só a fábrica muda.

5.4 Builder (Construtor)

- **Problema.** Os DTOs `JogoDTO` e `TimeDTO` têm vários campos. Um construtor longo é difícil de ler.
- **Onde aparece.** A anotação `@Builder` do Lombok em `JogoDTO` e em `TimeDTO`. O `DTOfactory` usa os builders.
- **Por que serve.** Cada campo aparece com o seu nome no código. A leitura fica clara.

5.5 Repository (Repositório)

- **Problema.** O código de negócio não deve conter código de acesso ao banco.
- **Onde aparece.** As interfaces de `api/repository`: `UsuarioRepository`, `AssociadoRepository`, `JogoRepository`, `TimeRepository` e `EstadioRepository`. Todas estendem `JpaRepository`.
- **Como funciona.** O Spring Data cria a implementação. A equipe escreve só consultas específicas, como `findByDhJogoBetween` e `checkDuplicateUser`.

5.6 Outros mecanismos de apoio

- **DTO e Form.** As telas recebem `JogoDTO`, `LoteIngressoDTO` e `CadastroForm`. As telas não recebem entidades.
- **Injeção de dependência.** O Spring cria os objetos e os entrega às classes. Os serviços e as fábricas são beans do Spring.

6. Padrões de arquitetura

6.1 Arquitetura em camadas

O sistema separa o código em camadas. Cada camada tem uma responsabilidade (seção 4.1). Uma camada só chama a camada logo abaixo.

Por que a equipe escolheu. A mudança em uma tela não altera a regra de negócio. A mudança no banco não altera os controllers. Cada pessoa da equipe pode trabalhar em uma camada.

6.2 MVC (Model-View-Controller)

Parte do MVC	No projeto
Model	Entidades de <code>core/model</code> , DTOs de <code>core/dto</code> e forms de <code>core/form</code>
View	Templates Thymeleaf em <code>src/main/resources/templates</code>
Controller	Classes de <code>api/controller</code>

O Spring MVC faz o papel do *front controller*: o `DispatcherServlet` recebe todos os pedidos e os entrega ao controller correto. Esse componente pertence ao framework, não ao código da equipe.

7. Interface gráfica

A interface é **web**. O Spring monta as páginas no servidor com Thymeleaf. O navegador mostra o HTML, o CSS e o JavaScript.

7.1 Telas

Tela	Endereço	Arquivo	Acesso	Conteúdo
Início	/	<code>home.html</code>	Todos	Só a barra de navegação
Login	/login	<code>login.html</code>	Todos	Formulário de login
Cadastro	/cadastro	<code>cadastro.html</code>	Todos	Formulário de cadastro
Jogos	/jogos	<code>jogos.html</code>	Usuário logado	Próximo jogo, filtro por mês e lista de jogos
Ingressos	/ingressos?jogoId=	<code>ingressos/lista.html</code>	Usuário logado	Lotes do jogo
Perfil	/perfil	<code>perfil.html</code>	USER e ADMIN	Mensagem de boas-vindas
Admin	/admin	<code>admin.html</code>	Só ADMIN	Mensagem de boas-vindas

A barra de navegação é um fragmento reutilizável (`fragments/navbar.html`). Todas as telas a incluem com `th:replace`.

7.2 Componentes básicos

Componente	Onde aparece
Botão (<code>button</code>)	Avatar da barra, Sair , Entrar , Cadastrar , meses do calendário, Comprar
Botão-link (<code>a.btn-cta</code>)	Entrar na barra, botão do próximo jogo
Campo de texto	Usuário (login e cadastro), CPF
Campo de senha	Login e cadastro
Campo de e-mail	Cadastro
Campo oculto	<code>loteId</code> no formulário de compra
Rótulo (<code>label</code>)	Todos os campos dos formulários
Formulário (<code>form</code>)	Login, cadastro, logout e compra
Link	Voltar , Cadastre-se , Entrar , Saiba Mais
Menu suspenso	Menu do avatar
Cartões	Próximo jogo, jogos da lista, lotes
Imagem	Escudos dos times
Mensagem de aviso	"Usuário ou senha inválidos." e "Você saiu da sua conta."

7.3 Eventos

Evento	Onde	O que acontece
onclick no botão do avatar	navbar.html	A função <code>toggleProfileMenu(event)</code> abre e fecha o menu suspenso.
click no documento	navbar.html	O menu suspenso fecha quando o usuário clica fora dele.
DOMContentLoaded	jogos.html	A página liga o evento de clique aos botões de mês e mostra os jogos do mês ativo (setembro, no código).
click nos botões de mês	jogos.html	O botão clicado fica ativo. Os cartões somem com animação de 150 ms. Só os jogos do mês escolhido voltam a aparecer. Sem jogos, a página mostra "Nenhum jogo encontrado para este mês."
submit do formulário de login	login.html	O navegador envia <code>POST /login</code> . O Spring Security autentica.
submit do formulário de cadastro	cadastro.html	O navegador envia <code>POST /cadastro</code> .
submit do formulário de logout	navbar.html	O navegador envia <code>POST /logout</code> .

7.4 Layouts

O CSS usa três técnicas de layout:

- **Flexbox** nas barras, nos formulários, nos cartões e nas listas (`navbar.css`, `auth.css`, `jogos.css`, `ingressos.css`).
- **CSS Grid** nos cartões de jogo (`.jogo-card`, `.confronto-verus`) e nos cartões de lote (`.lote-card`).
- **Media queries** em `navbar.css` (largura até 900 px) e em `ingressos.css` (largura até 640 px).

O CSS fica em `static/css`: `main.css`, `navbar.css`, `auth.css`, `jogos.css` e `ingressos.css`.

7.5 Navegação

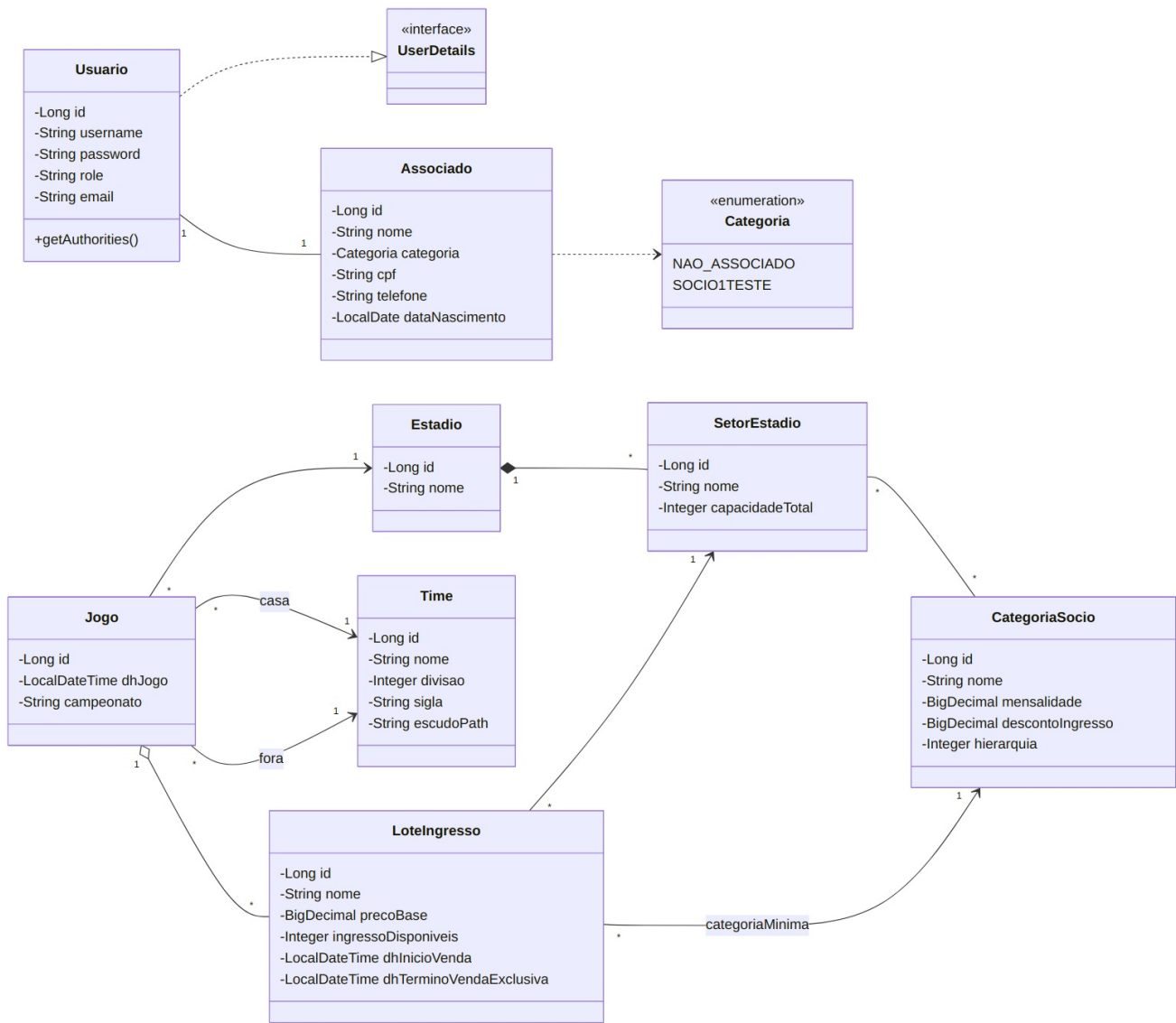
A Figura 5 (seção 8) mostra como o usuário passa de uma tela para outra. Quem não está logado e abre `/jogos` ou `/ingressos` vai para a tela de login.

8. Diagramas UML

Figura	Tipo	Objetivo	Seção relacionada
1	Diagrama de classes (domínio)	Mostrar as entidades e os relacionamentos	3 e 4
2	Diagrama de classes (validação)	Mostrar os padrões Chain of Responsibility, Template Method e Factory	5
3	Diagrama de camadas	Mostrar a arquitetura em camadas	4 e 6
4	Diagrama de sequência (lotes)	Mostrar o fluxo de <code>GET /ingressos</code>	4.3
5	Diagrama de navegação (estados)	Mostrar o fluxo entre as telas	7
6	Diagrama de sequência (cadastro)	Mostrar o fluxo de <code>POST /cadastro</code>	3 e 5

Todos os diagramas vêm do código real. O diagrama de domínio não mostra as entidades `Endereco`, `Localidade`, `BeneficioSocio` e `Noticia`, porque nenhuma outra classe usa essas entidades.

Figura 1 — Classes do domínio



A associação entre **SetorEstadio** e **CategoriaSocio** é de muitos para muitos. No código, cada lado declara a sua própria lista com `@ManyToMany`.

Figura 2 — Classes da validação de lotes

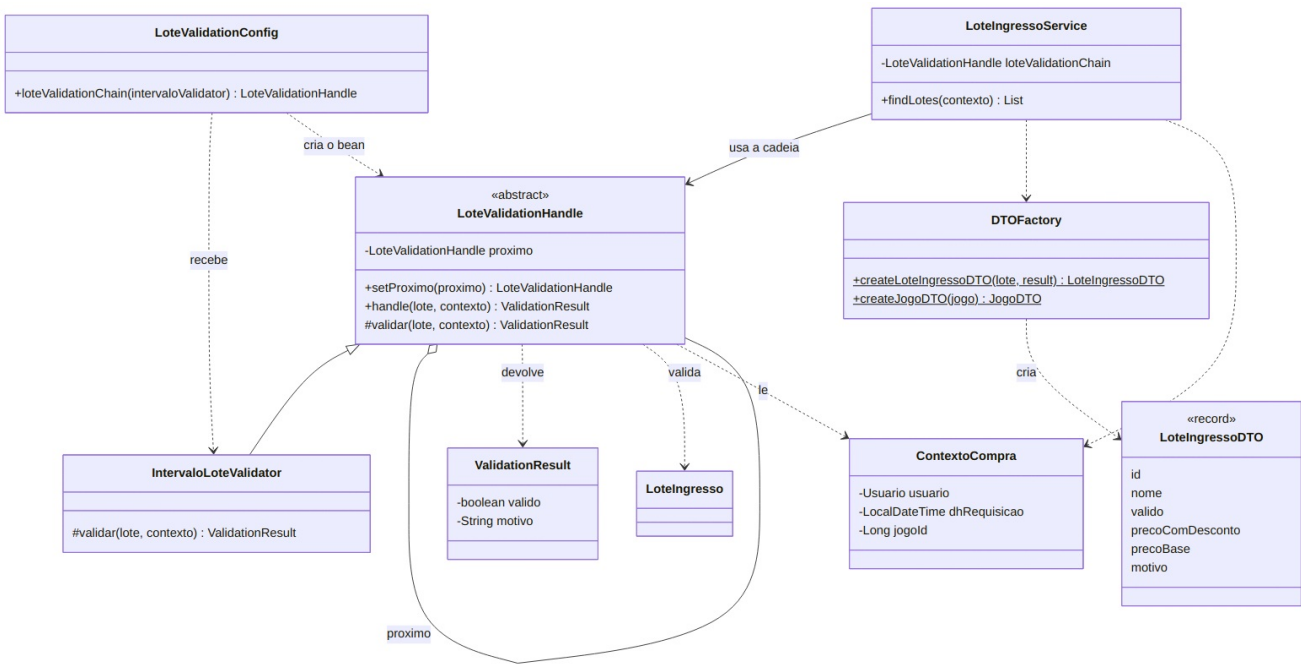


Figura 3 — Camadas

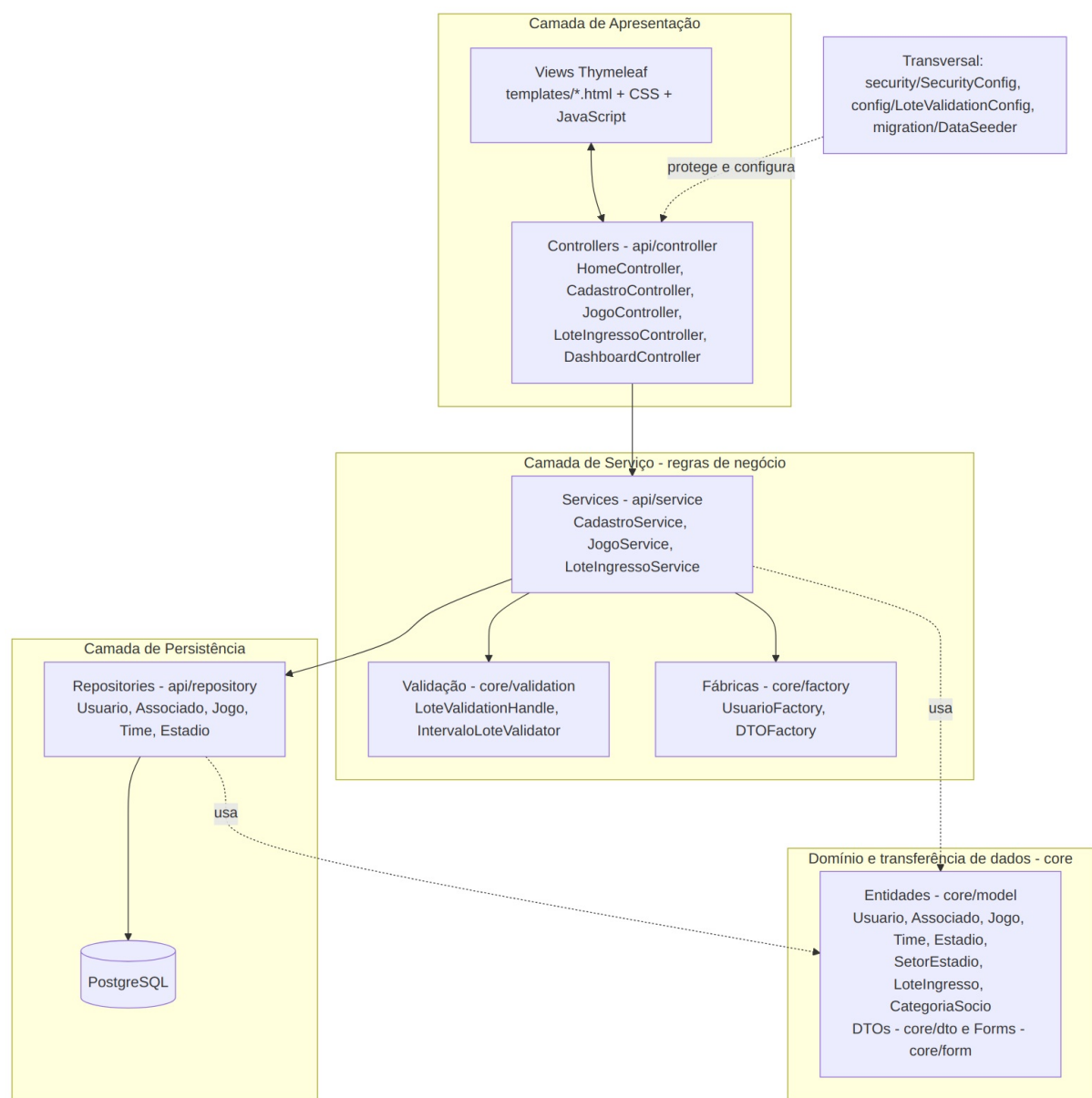


Figura 4 — Sequência: listar lotes de um jogo

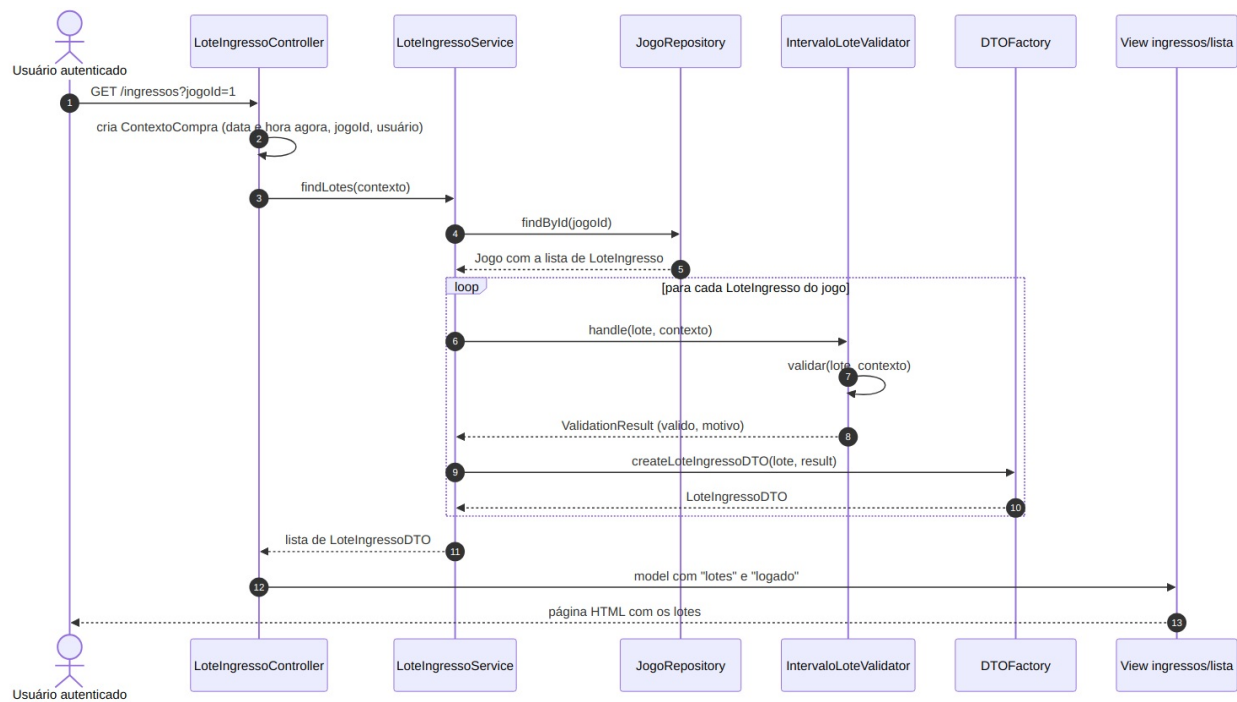


Figura 5 — Navegação entre as telas

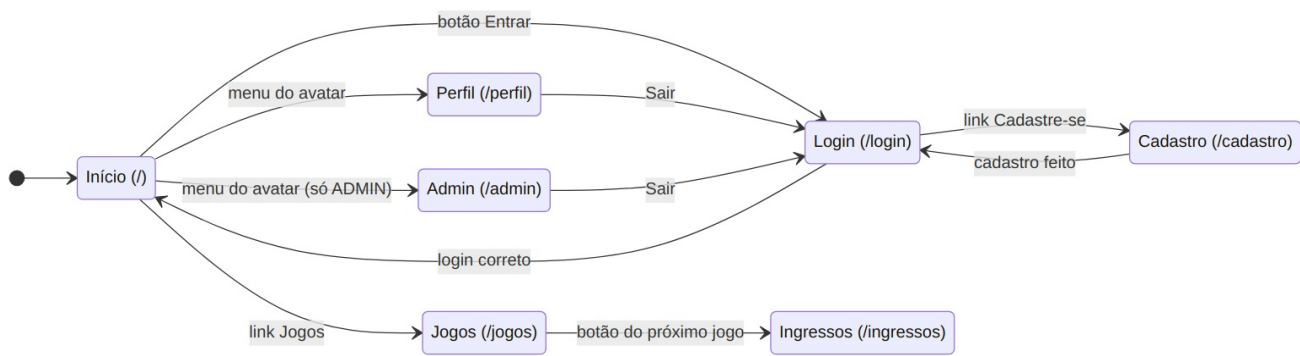
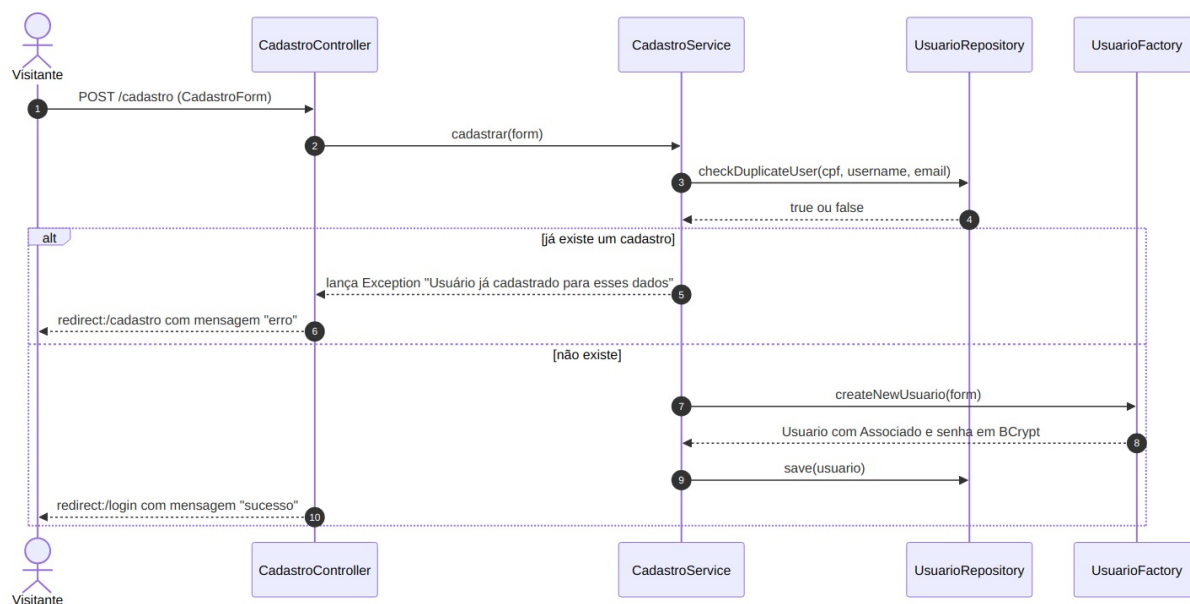


Figura 6 — Sequência: cadastro de usuário



O fluxo da Figura 6 mostra o comportamento atual: o controller chama o serviço sem consultar o resultado da validação do formulário (seção 11).

9. Decisões de projeto

Cada decisão tem o contexto, a escolha, o motivo e a consequência.

N.º	Decisão	Motivo	Consequência
D1	Usar arquitetura em camadas com Spring MVC	Separar tela, regra e dados. Seguir a estrutura que o Spring Boot já oferece.	Nenhum controller acessa o banco. Cada camada muda sem afetar as outras.
D2	Validar o lote com Chain of Responsibility	As regras de venda vão crescer. Cada regra deve ficar em uma classe.	Uma regra nova exige só uma classe nova e uma ligação na cadeia. Hoje existe uma regra.
D3	Fixar o fluxo da cadeia com Template Method em <code>LoteValidationHandle</code>	Todas as regras precisam seguir o mesmo fluxo.	O método <code>handle()</code> é final. As subclasses implementam só <code>validar()</code> .
D4	Criar <code>Usuario</code> e DTOs em fábricas	A criação tem vários passos, como a criptografia da senha.	O código de criação fica em um único lugar.
D5	Enviar DTOs e Forms para as telas, não entidades	A tela não precisa conhecer a estrutura do banco.	Mudar uma entidade não quebra a tela.
D6	Usar <code>@Builder</code> do Lombok em <code>JogoDTO</code> e <code>TimeDTO</code>	Esses DTOs têm vários campos.	A criação do objeto fica legível.
D7	Acessar o banco com Spring Data JPA	Evitar código repetido de acesso a dados.	Os repositórios são interfaces. A equipe escreve só consultas específicas.
D8	Autenticar com Spring Security, BCrypt e dois papéis (<code>USER</code> , <code>ADMIN</code>)	Proteger as telas e guardar senhas com segurança.	A classe <code>Usuario</code> implementa <code>UserDetails</code> . Isso simplifica o código, mas liga a entidade ao Spring Security.
D9	Montar as páginas no servidor com Thymeleaf e usar um fragmento de navegação	Reutilizar a barra de navegação em todas as telas. Manter o JavaScript pequeno.	Cada tela inclui o fragmento com <code>th:replace</code> . O JavaScript fica dentro dos arquivos HTML.
D10	Usar PostgreSQL, <code>ddl-auto=update</code> , variáveis de ambiente e <code>DataSeeder</code>	Facilitar o desenvolvimento. Não guardar senhas no código.	O banco se atualiza sozinho. A demonstração começa com dados prontos.

Desvio conhecido (D11). A classe `InjectionProvider` guarda os repositórios em campos estáticos. `CadastroService`, `JogoService` e `LoteIngressoService` usam esses campos. Essa prática foge da injeção de dependência da seção 5.6, porque dificulta testes e esconde as dependências. As classes `DataSeeder`, `JogoController` e `LoteIngressoController` já usam injeção por construtor. O próximo passo é aplicar o mesmo estilo nos serviços.

10. Testes

10.1 Testes automáticos

Existe um teste automático: `SociotorcedorApplicationTests.contextLoads()`. Ele verifica se o contexto do Spring inicia. O teste precisa do PostgreSQL e das variáveis de ambiente.

10.2 Testes manuais

O arquivo `testes/README.md` registra 14 casos manuais de cadastro, login e jogos:

Resultado	Quantidade
Aprovado	4
Falha	7
Regra a confirmar	2
Comportamento observado	1

10.3 Causas identificadas no código

A leitura do código mostra três causas para as falhas:

1. O `CadastroController` recebe o `BindingResult`, mas não consulta o resultado. As anotações de validação do `CadastroForm` não bloqueiam o cadastro.
2. Os templates `cadastro.html` e `login.html` não mostram as mensagens `erro` e `sucesso` enviadas pelo controller.
3. A consulta `checkDuplicateUser` compara `associado.nome` com o `username`. Ela não compara o campo `Usuario.username`.

10.4 Testes previstos

Estes testes ainda **não existem**. A classe `IntervaloLoteValidator` é a primeira candidata, porque não depende do banco.

Caso	Entrada	Resultado esperado
Venda ainda não começou	Horário do pedido antes de <code>dhInicioVenda</code>	<code>ValidationResult</code> inválido, com o motivo
Venda aberta	Horário entre <code>dhInicioVenda</code> e o horário do jogo	<code>ValidationResult</code> válido
Jogo já aconteceu	Horário do pedido depois do jogo	<code>ValidationResult</code> inválido, com o motivo

11. Conclusão

O sistema tem uma base de código organizada em camadas. Ele usa cinco padrões de projeto e o padrão MVC. A interface web cobre cadastro, login, calendário de jogos e lotes de ingresso.

Limitações conhecidas

- A compra de ingresso, a associação de sócios, os descontos e os benefícios ainda não funcionam (seção 3.3).
- A cadeia de validação tem uma única regra.
- O formulário de cadastro não bloqueia dados inválidos (seção 10.3).
- A rota `/jogos` exige login, mas a barra de navegação mostra o link **Jogos** para visitantes.
- As telas Início, Perfil e Admin têm pouco conteúdo.
- O código usa três estilos de injeção de dependência (seção 9, D11).

Próximos passos

1. Consultar o `BindingResult` no cadastro e mostrar as mensagens nas telas.
2. Corrigir a consulta `checkDuplicateUser`.
3. Criar o endpoint de compra e novas regras na cadeia (categoria mínima e estoque de ingressos).
4. Escrever os testes unitários da seção 10.4.
5. Trocar o uso do `InjectionProvider` por injeção por construtor.

12. Glossário

Termo	Significado
-------	-------------

Associado	Pessoa ligada a uma categoria de sócio.
Bean	Objeto criado e gerenciado pelo Spring.
Camada	Grupo de classes com a mesma responsabilidade.
Controller	Classe que recebe o pedido do navegador.
DTO	Objeto simples que leva dados entre as camadas.
Entidade	Classe ligada a uma tabela do banco.
Lote	Grupo de ingressos de um jogo, com preço e período de venda.
Padrão de arquitetura	Modelo de organização do sistema inteiro.
Padrão de projeto	Modelo de solução para um problema de classes e objetos.
Repositório	Interface que lê e grava dados no banco.
Serviço	Classe que aplica as regras de negócio.

Documentation of the Sócio-Torcedor Management System

Project of the Object-Oriented Programming Language (LPOO) course — Computer Science, IFSul.

Team: Allan Theodoro, Bernardo Zavistanovicz Deters, Olavo Appel Neto and Thomas Cansian dos Santos.

Versions of this document: Português · English (this file) · Leitura Fácil

Writing rules. This text follows the main rules of ASD-STE100: short sentences, active voice, simple present tense, and one topic in each sentence. The names of classes, files and tools are technical names. They stay in their original form. The text was not checked with the official STE dictionary.

1. Introduction

This document describes the Sócio-Torcedor Management System. It shows what the system does. It shows how the code is organized. It gives the reason for each design decision.

The document describes the code **as it is today**. Section 3.3 and Section 11 show what does not work yet.

The UML diagrams are in Section 8. The source files of the diagrams are in the folder `docs/diagramas/`.

2. System purpose

The system helps a football club to manage its supporters (sócio-torcedores). The project proposal has four areas:

1. **Member management:** sign-up of new members and change of plan.
2. **Categories and discounts:** member classes, with prices and discounts for each class.
3. **Ticket control:** prices for each championship and control of ticket sales for the games.
4. **Benefit program:** control of gifts and special experiences.

In this delivery, the system does four things. It signs up users. It logs users in. It shows a calendar of games. It shows the ticket batches of each game. The other areas are not ready (Section 3.3).

The project also shows four skills for the course: documentation, design patterns, graphical user interface, and architecture patterns.

3. System description

3.1 Technologies

Item	Technology
Language	Java 26
Main framework	Spring Boot 4.1.1
Build tool	Maven (with <code>mvnw</code>)
Web layer	Spring MVC and Thymeleaf
Persistence	Spring Data JPA (Hibernate)
Security	Spring Security, with BCrypt passwords
Validation	Bean Validation
Less repeated code	Lombok
Database	PostgreSQL
Interface	HTML, CSS and JavaScript

3.2 Features that work

- **User sign-up.** The visitor gives a user name, an e-mail address, a password and a CPF. The system makes a `Usuario` with

an `Associado` in the category `NAO_ASSOCIADO`.

- **Login and logout.** The system uses a login form. There are two roles: `USER` and `ADMIN`.
- **Game calendar.** The page shows the games of the year. It shows the next game in a large card. It has a filter for each month.
- **Ticket batches.** The page shows the batches of one game. Each batch is valid or not valid. The page shows the reason.
- **Profile and Admin pages.** Each page shows the name of the user. Only the role `ADMIN` can open the Admin page.
- **Initial data.** The class `DataSeeder` makes the user `admin`, six teams, two stadiums, six games and four batches for the next game.

3.3 Features that are not ready

Item	State in the code
Ticket purchase	The button Comprá exists. The address <code>/ingressos/comprar</code> has no controller. The method <code>IngressoService.comprarIngresso()</code> is empty.
Become a member	<code>SocioTorcedorController</code> is in comments. <code>SocioTorcedorService.tornarseSocio()</code> exists, but no class calls it.
Discount for each category	The price with discount is the same as the base price (in <code>DTOfactory</code>).
Benefit program	The entity <code>BeneficioSocio</code> exists, but no class uses it.
Button Saiba Mais	The link goes to <code>#</code> .

3.4 How to start the system

1. Install JDK 26 and PostgreSQL.
2. Make an empty database.
3. Set the environment variables `DB_NAME`, `DB_USER` and `DB_PASSWORD`.
4. In the folder `sociotorcedor/`, run `./mvnw spring-boot:run`.
5. Open `http://localhost:8080`.

The system connects to the database at `localhost:5432`. At the first start, the `DataSeeder` makes the initial data. The demonstration user is `admin`, with the password `admin`.

3.5 Data storage

The system keeps the data in PostgreSQL. The property `spring.jpa.hibernate.ddl-auto=update` tells Hibernate to make and change the tables. The system does not use migration files.

4. Architecture

The system uses a **layered architecture**. For the web part, it uses the Spring **MVC** pattern. Section 6 gives the reasons. This section shows the structure.

4.1 Layers

Layer	Packages	Job
Presentation	<code>templates/</code> , <code>static/</code> , <code>api/controller</code>	Gets the request from the browser. Calls the service. Selects the page.
Service	<code>api/service</code> , <code>core/validation</code> , <code>core/factory</code>	Applies the business rules. Checks the data. Makes objects.
Persistence	<code>api/repository</code>	Reads and writes the data in the database.
Domain and data transfer	<code>core/model</code> , <code>core/dto</code> , <code>core/form</code>	Defines the entities and the objects that go between the layers.
Cross-cutting	<code>security</code> , <code>config</code> , <code>migration</code>	Sets the security, the validation chain and the initial data.

4.2 How the layers communicate

- The browser talks only to the controllers.
- The controllers talk only to the services. No controller uses the database.
- The services use the repositories, the factories and the validation chain.
- The repositories talk to PostgreSQL.
- All layers use the domain classes.

Figure 3 (Section 8) shows this structure.

4.3 Example of a request

The user opens `/ingressos?jogoId=1`. The `LoteIngressoController` makes a `ContextoCompra`. Then it calls the `LoteIngressoService`. The service gets the game from the `JogoRepository`. The service checks each batch with the validation chain. The `DTOfactory` makes a `LoteIngressoDTO` for each batch. The controller sends the list to the page `ingressos/lista`. Figure 4 shows this flow.

5. Design patterns used

The team uses five design patterns. A class or a group of classes implements each pattern.

5.1 Chain of Responsibility

- **Problem.** The rules for the sale of a batch will grow. If all rules are in one method, the method has a large `if` block. It is difficult to change.
- **Where it is.** `LoteValidationHandle`, `IntervaloLoteValidator`, `LoteValidationConfig` and `LoteIngressoService`.
- **How it works.** Each rule is a class. Each class gets the batch and checks it. Then it sends the batch to the next class in the chain. The chain stops at the first rule that fails. `LoteValidationConfig` builds the chain. `LoteIngressoService` calls only the first class.
- **Current state.** The chain has **one rule**: `IntervaloLoteValidator`. This rule checks that the time of the request is inside the sale period of the batch. To add a rule, the team writes a new class and connects it with `setProximo()`. The model already has data for new rules, such as `categoriaMinima` and `ingressoDisponiveis`. These rules do not exist yet.
- **Why it is correct.** `LoteIngressoService` does not change when a new rule is added. This follows the open/closed principle.

5.2 Template Method

- **Problem.** All rules in the chain must follow the same flow: check, stop if the check fails, call the next rule. A rule must not repeat this flow.
- **Where it is.** `LoteValidationHandle`.
- **How it works.** The method `handle()` is `final`. It sets the order of the steps. The method `validar()` is abstract. Each subclass implements only `validar()`.
- **Why it is correct.** The flow of the chain is in one place. No subclass can change it.

5.3 Factory (simple factory)

- **Problem.** Some objects need many steps to make. If these steps are in many places, the code has repeated parts and errors.
- **Where it is.**
 - `UsuarioFactory.createNewUsuario()` makes a `Usuario` with an `Associado`. It sets the category `NAO_ASSOCIADO` and the role `USER`. It encrypts the password.
 - `DTOfactory` changes entities into DTOs (`createJogoDTO` and `createLoteIngressoDTO`).
- **Why it is correct.** `CadastroService` and `JogoService` do not know the details of the object creation. If the creation changes, only the factory changes.

5.4 Builder

- **Problem.** The DTOs `JogoDTO` and `TimeDTO` have many fields. A long constructor is difficult to read.
- **Where it is.** The Lombok annotation `@Builder` in `JogoDTO` and in `TimeDTO`. `DTOfactory` uses the builders.
- **Why it is correct.** Each field has its name in the code. The code is easy to read.

5.5 Repository

- **Problem.** The business code must not contain code for database access.
- **Where it is.** The interfaces in `api/repository`: `UsuarioRepository`, `AssociadoRepository`, `JogoRepository`, `TimeRepository` and `EstadioRepository`. All of them extend `JpaRepository`.
- **How it works.** Spring Data makes the implementation. The team writes only special queries, such as `findByDhJogoBetween` and `checkDuplicateUser`.

5.6 Other support mechanisms

- **DTO and Form.** The pages get `JogoDTO`, `LoteIngressoDTO` and `CadastroForm`. The pages do not get entities.
- **Dependency injection.** Spring makes the objects and gives them to the classes. The services and the factories are Spring beans.

6. Architecture patterns

6.1 Layered architecture

The system divides the code into layers. Each layer has one job (Section 4.1). A layer calls only the layer below it.

Reason for the choice. A change to a page does not change the business rules. A change to the database does not change the controllers. Each team member can work on one layer.

6.2 MVC (Model-View-Controller)

MVC part	In the project
Model	Entities in <code>core/model</code> , DTOs in <code>core/dto</code> and forms in <code>core/form</code>
View	Thymeleaf templates in <code>src/main/resources/templates</code>
Controller	Classes in <code>api/controller</code>

Spring MVC has the role of the *front controller*. The `DispatcherServlet` gets all requests. It sends each request to the correct controller. This component belongs to the framework. It is not code of the team.

7. Graphical user interface

The interface is a **web** interface. Spring builds the pages on the server with Thymeleaf. The browser shows the HTML, the CSS and the JavaScript.

7.1 Pages

Page	Address	File	Access	Content
Home	/	<code>home.html</code>	Everyone	Only the navigation bar
Login	/login	<code>login.html</code>	Everyone	Login form
Sign-up	/cadastro	<code>cadastro.html</code>	Everyone	Sign-up form
Games	/jogos	<code>jogos.html</code>	Signed-in user	Next game, month filter and list of games
Tickets	/ingressos?jogoId=	<code>ingressos/lista.html</code>	Signed-in user	Batches of the game
Profile	/perfil	<code>perfil.html</code>	USER and ADMIN	Welcome message
Admin	/admin	<code>admin.html</code>	ADMIN only	Welcome message

The navigation bar is a fragment that all pages use (`fragments/navbar.html`). Each page includes it with `th:replace`.

7.2 Basic components

Component	Where it is
Button (<code>button</code>)	Avatar in the bar, Sair , Entrar , Cadastrar , calendar months, Comprar
Link button (<code>a.btn-cta</code>)	Entrar in the bar, button of the next game
Text field	User name (login and sign-up), CPF
Password field	Login and sign-up
E-mail field	Sign-up
Hidden field	<code>loteId</code> in the purchase form
Label (<code>label</code>)	All fields of the forms
Form (<code>form</code>)	Login, sign-up, logout and purchase
Link	Voltar , Cadastre-se , Entrar , Saiba Mais
Drop-down menu	Avatar menu
Cards	Next game, games in the list, batches
Image	Team crests
Message	"Usuário ou senha inválidos." and "Você saiu da sua conta."

7.3 Events

Event	Where	What happens
onclick on the avatar button	navbar.html	The function <code>toggleProfileMenu(event)</code> opens and closes the drop-down menu.
click on the document	navbar.html	The drop-down menu closes when the user clicks outside it.
DOMContentLoaded	jogos.html	The page connects the click event to the month buttons. It shows the games of the active month (September, in the code).
click on the month buttons	jogos.html	The clicked button becomes active. The cards fade out in 150 ms. Only the games of the selected month appear again. If there are no games, the page shows "Nenhum jogo encontrado para este mês."
submit of the login form	login.html	The browser sends <code>POST /login</code> . Spring Security does the authentication.
submit of the sign-up form	cadastro.html	The browser sends <code>POST /cadastro</code> .
submit of the logout form	navbar.html	The browser sends <code>POST /logout</code> .

7.4 Layouts

The CSS uses three layout methods:

- **Flexbox** in the bars, the forms, the cards and the lists (`navbar.css`, `auth.css`, `jogos.css`, `ingressos.css`).
- **CSS Grid** in the game cards (`.jogo-card`, `.confronto-verus`) and in the batch cards (`.lote-card`).
- **Media queries** in `navbar.css` (width up to 900 px) and in `ingressos.css` (width up to 640 px).

The CSS files are in `static/css`: `main.css`, `navbar.css`, `auth.css`, `jogos.css` and `ingressos.css`.

7.5 Navigation

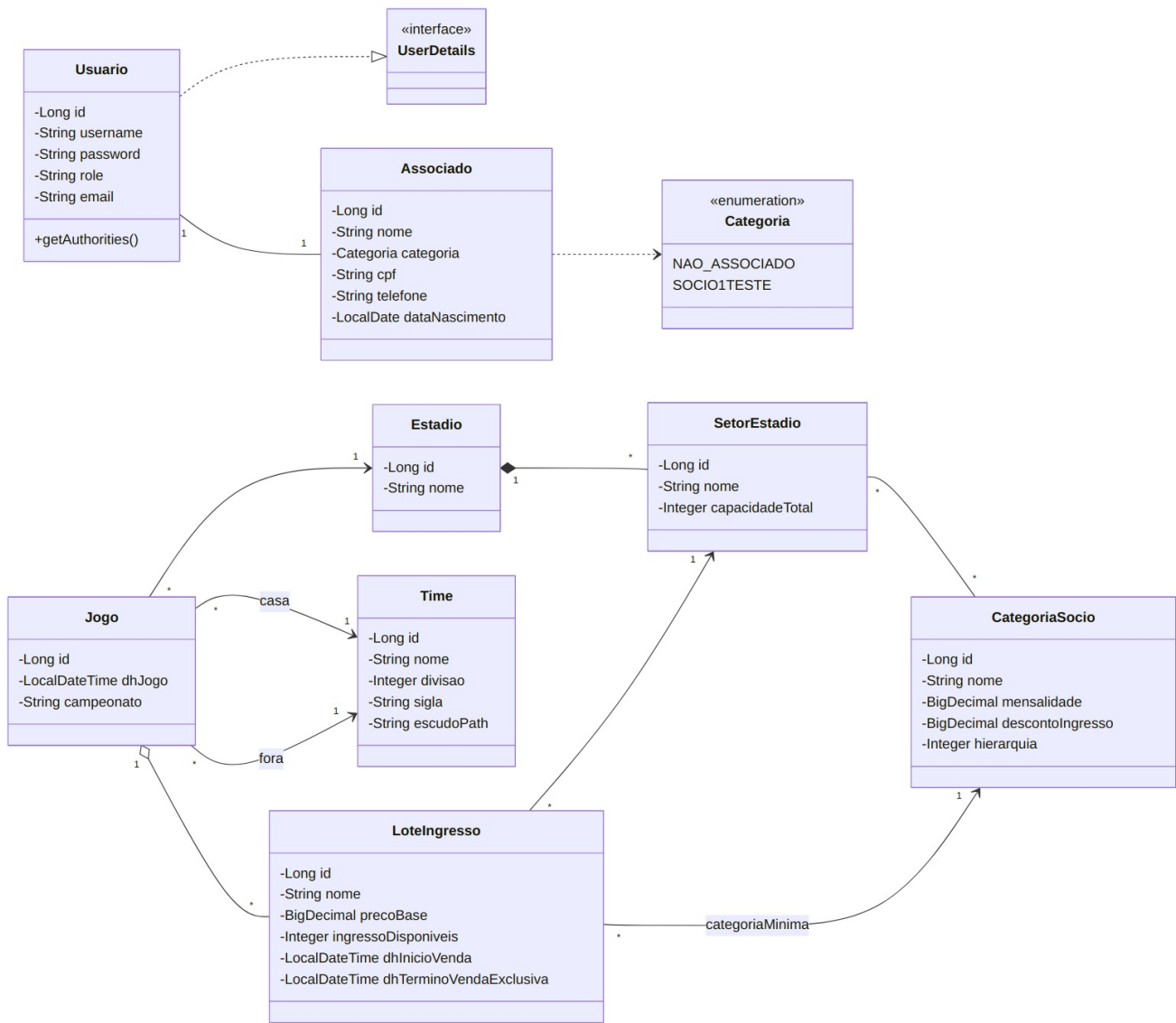
Figure 5 (Section 8) shows how the user goes from one page to another. A user who is not signed in and opens `/jogos` or `/ingressos` goes to the login page.

8. UML diagrams

Figure	Type	Purpose	Related section
1	Class diagram (domain)	Show the entities and their relations	3 and 4
2	Class diagram (validation)	Show the patterns Chain of Responsibility, Template Method and Factory	5
3	Layer diagram	Show the layered architecture	4 and 6
4	Sequence diagram (batches)	Show the flow of <code>GET /ingressos</code>	4.3
5	Navigation diagram (states)	Show the flow between the pages	7
6	Sequence diagram (sign-up)	Show the flow of <code>POST /cadastro</code>	3 and 5

All diagrams come from the real code. The domain diagram does not show the entities `Endereco`, `Localidade`, `BeneficioSocio` and `Noticia`, because no other class uses them.

Figure 1 — Domain classes



The association between `SetorEstadio` and `CategoriaSocio` is many-to-many. In the code, each side has its own list with `@ManyToMany`.

Figure 2 — Classes of the batch validation

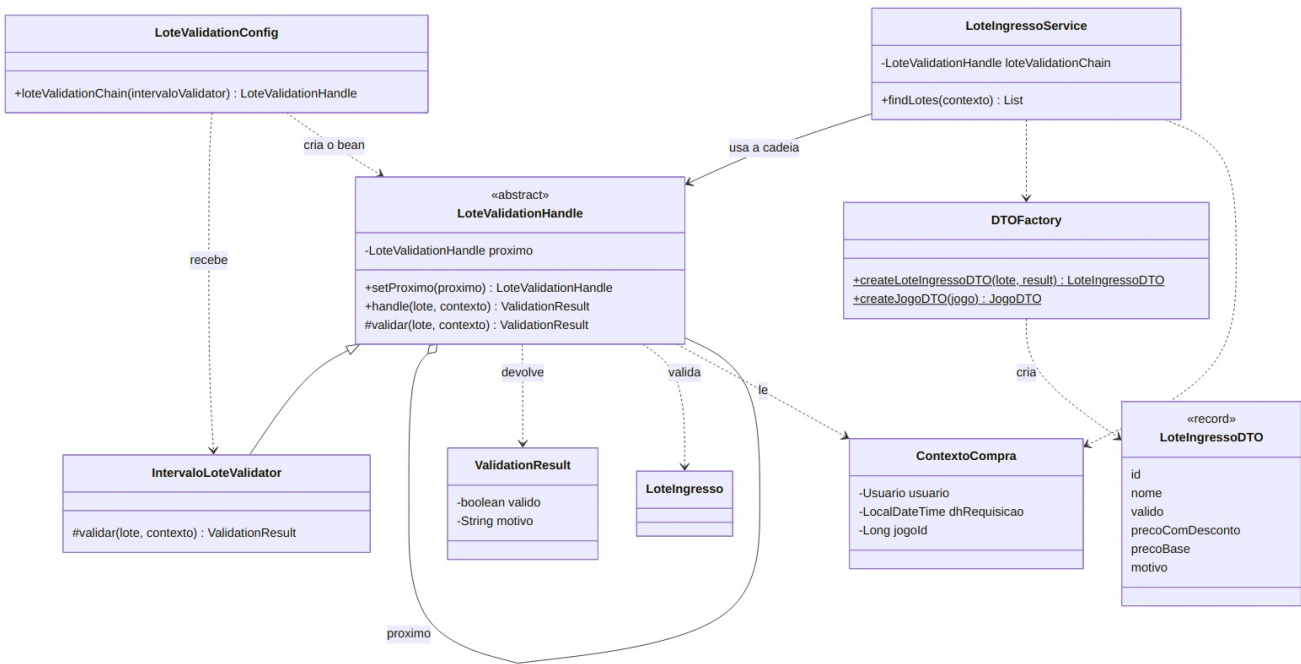


Figure 3 — Layers

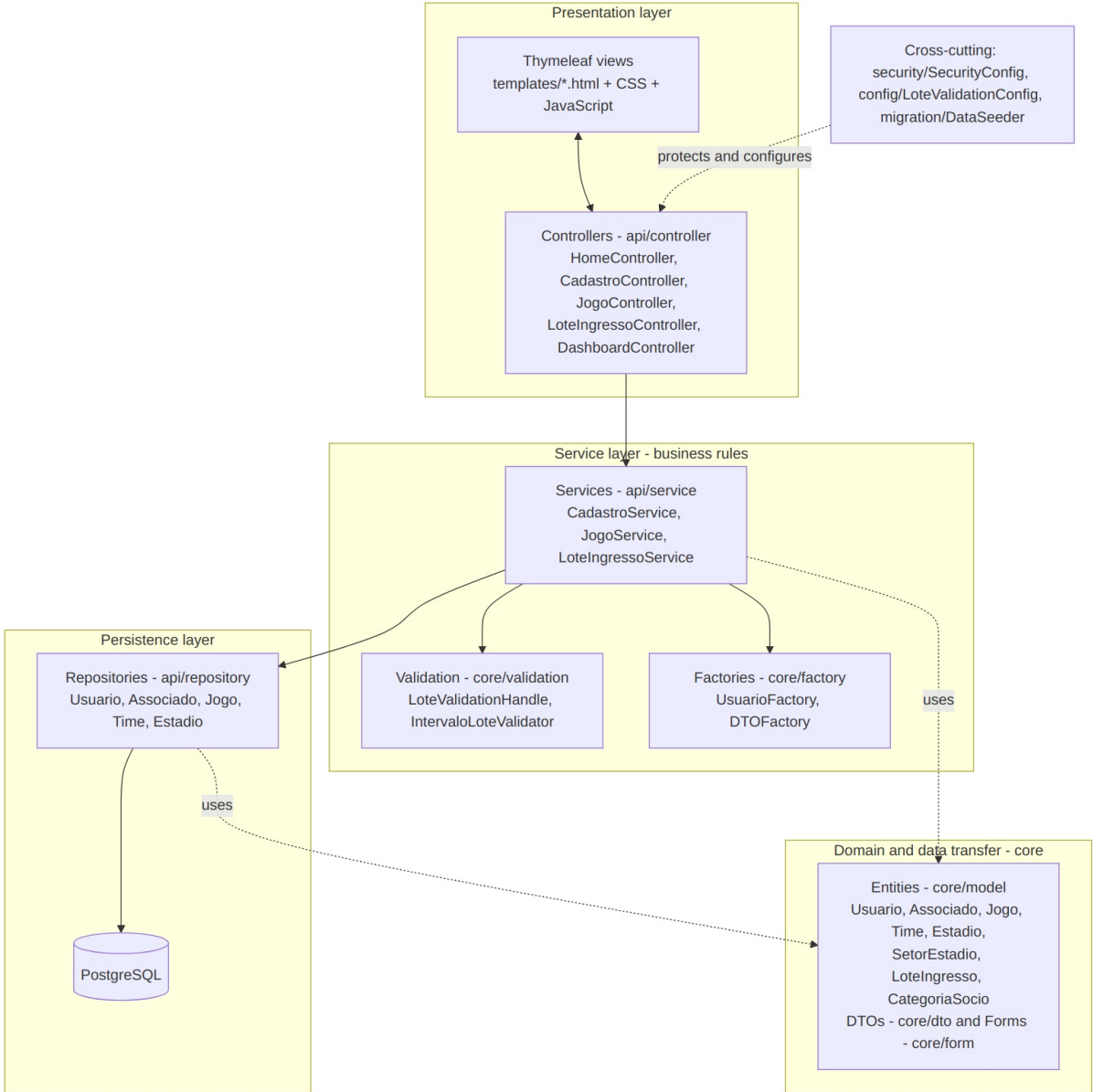


Figure 4 — Sequence: list the batches of a game

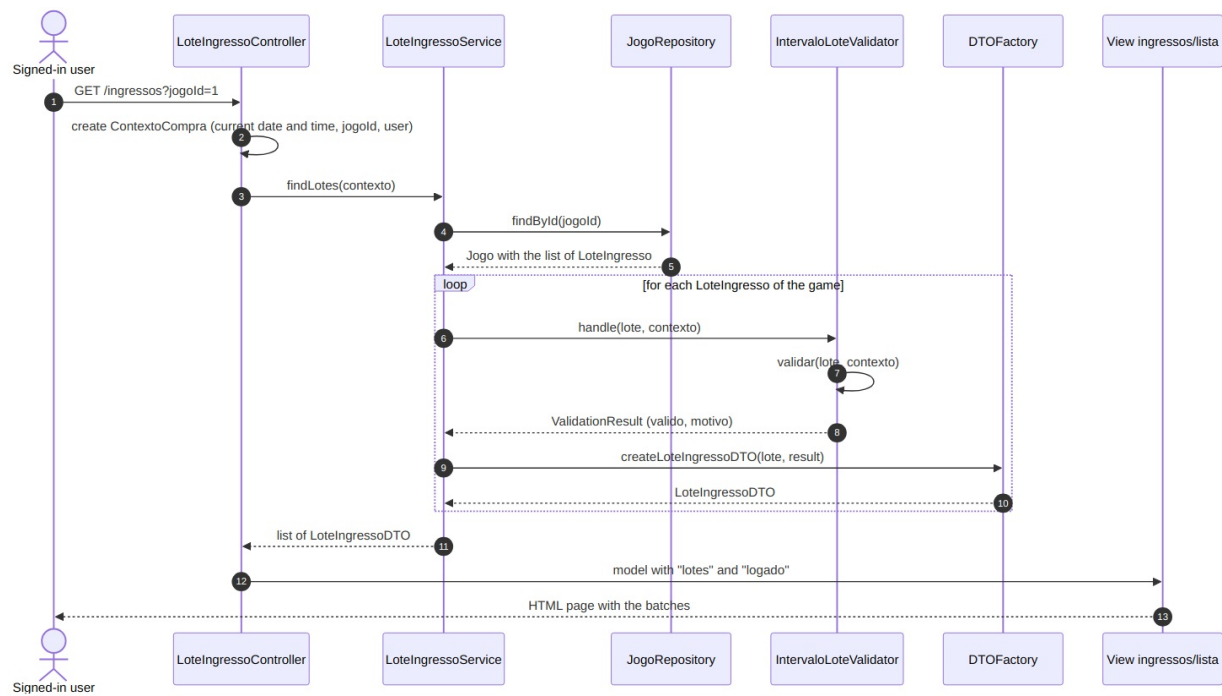


Figure 5 — Navigation between the pages

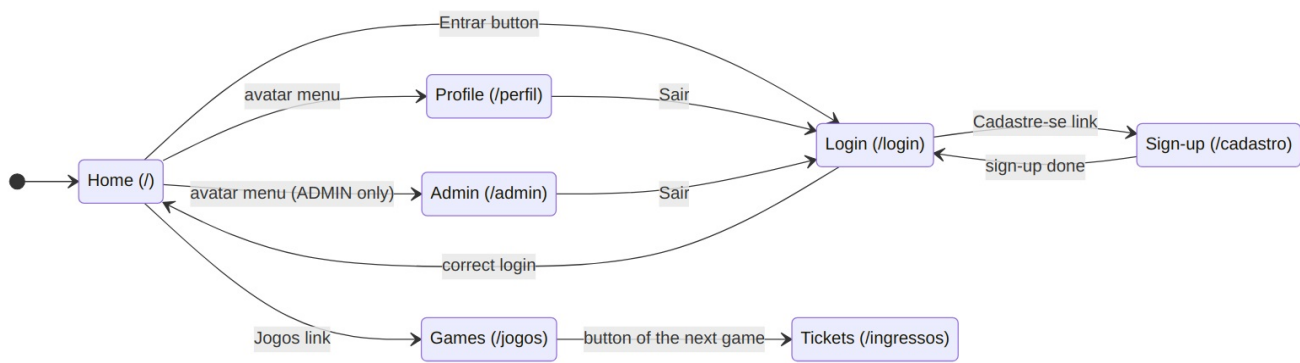


Figure 6 — Sequence: user sign-up

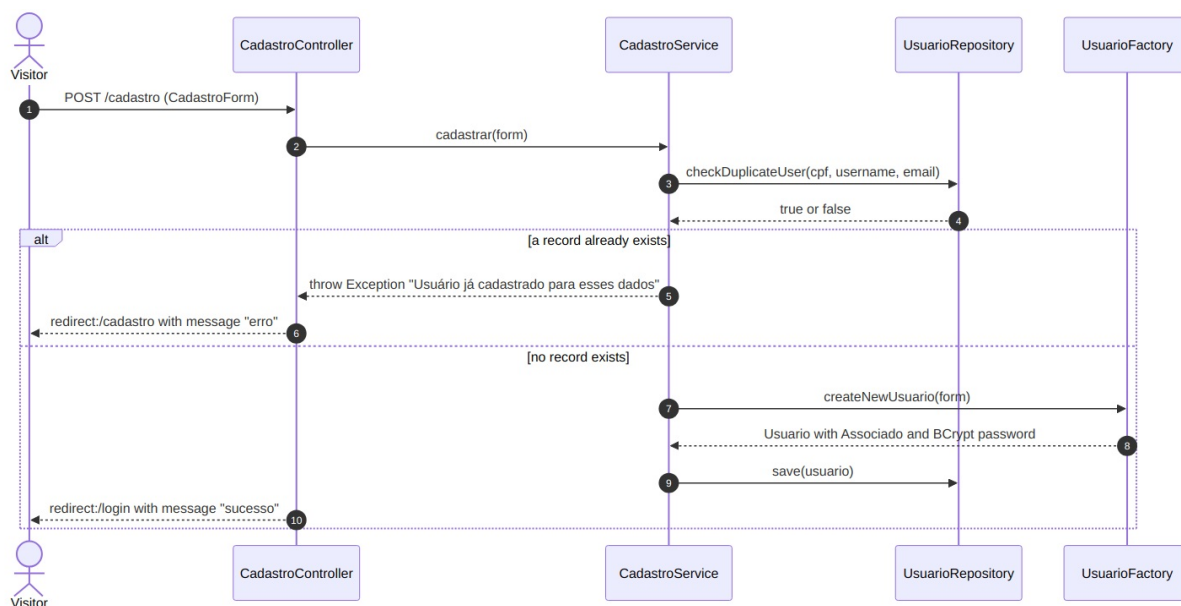


Figure 6 shows the current behavior: the controller calls the service and does not read the result of the form check (Section 11).

9. Design decisions

Each decision has a reason and a result.

No.	Decision	Reason	Result
D1	Use a layered architecture with Spring MVC	Keep the page, the rule and the data apart. Follow the structure that Spring Boot gives.	No controller uses the database. Each layer can change without a change in the other layers.
D2	Check the batch with Chain of Responsibility	The sale rules will grow. Each rule must be in one class.	A new rule needs one new class and one link in the chain. Today there is one rule.
D3	Fix the flow of the chain with Template Method in <code>LoteValidationHandle</code>	All rules must follow the same flow.	The method <code>handle()</code> is <code>final</code> . The subclasses implement only <code>validar()</code> .
D4	Make <code>Usuario</code> and the DTOs in factories	The creation has many steps, such as the password encryption.	The creation code is in one place.
D5	Send DTOs and Forms to the pages, not entities	The page does not need to know the structure of the database.	A change to an entity does not break the page.
D6	Use <code>@Builder</code> from Lombok in <code>JogoDTO</code> and <code>TimeDTO</code>	These DTOs have many fields.	The object creation is easy to read.
D7	Use Spring Data JPA for the database	Do not repeat the data access code.	The repositories are interfaces. The team writes only special queries.
D8	Use Spring Security, BCrypt and two roles (<code>USER</code> , <code>ADMIN</code>)	Protect the pages and keep the passwords safe.	The class <code>Usuario</code> implements <code>UserDetails</code> . This makes the code simple, but it connects the entity to Spring Security.
D9	Build the pages on the server with Thymeleaf and use a navigation fragment	Use the same navigation bar on all pages. Keep the JavaScript small.	Each page includes the fragment with <code>th:replace</code> . The JavaScript is inside the HTML files.
D10	Use PostgreSQL, <code>ddl-auto=update</code> , environment variables and <code>DataSeeder</code>	Make development easy. Do not keep passwords in the code.	The database updates itself. The demonstration starts with data.

Known deviation (D11). The class `InjectionProvider` keeps the repositories in static fields. `CadastroService`, `JogoService` and `LoteIngressoService` use these fields. This practice does not follow the dependency injection of Section 5.6. It makes tests difficult and it hides the dependencies. The classes `DataSeeder`, `JogoController` and `LoteIngressoController` already use constructor injection. The next step is to use the same style in the services.

10. Tests

10.1 Automatic tests

There is one automatic test: `SociotorcedorApplicationTests.contextLoads()`. It checks that the Spring context starts. The test needs PostgreSQL and the environment variables.

10.2 Manual tests

The file `testes/README.md` has 14 manual cases for sign-up, login and games:

Result	Number
Passed	4
Failed	7
Rule to confirm	2
Observed behavior	1

10.3 Causes found in the code

The code shows three causes for the failures:

1. `CadastroController` gets the `BindingResult`, but it does not read it. The validation annotations of `CadastroForm` do not stop the sign-up.
2. The templates `cadastro.html` and `login.html` do not show the messages `erro` and `sucesso` that the controller sends.
3. The query `checkDuplicateUser` compares `associado.nome` with the user name. It does not compare the field `Usuario.username`.

10.4 Planned tests

These tests do **not** exist yet. The class `IntervaloLoteValidator` is the first candidate, because it does not need the database.

Case	Input	Expected result
Sale did not start	Time of the request before <code>dhInicioVenda</code>	<code>ValidationResult</code> not valid, with the reason
Sale is open	Time between <code>dhInicioVenda</code> and the time of the game	<code>ValidationResult</code> valid
Game already happened	Time of the request after the game	<code>ValidationResult</code> not valid, with the reason

11. Conclusion

The system has a code base in layers. It uses five design patterns and the MVC pattern. The web interface covers sign-up, login, the game calendar and the ticket batches.

Known limits

- Ticket purchase, member sign-up, discounts and benefits do not work yet (Section 3.3).
- The validation chain has only one rule.
- The sign-up form does not stop wrong data (Section 10.3).
- The route `/jogos` needs a login, but the navigation bar shows the link **Jogos** to visitors.
- The pages Home, Profile and Admin have little content.
- The code uses three styles of dependency injection (Section 9, D11).

Next steps

1. Read the `BindingResult` in the sign-up and show the messages on the pages.
2. Correct the query `checkDuplicateUser`.
3. Make the purchase endpoint and new rules in the chain (minimum category and ticket stock).
4. Write the unit tests of Section 10.4.
5. Replace the use of `InjectionProvider` with constructor injection.

12. Glossary

Term	Meaning
------	---------

Asociado	A person who has a member category.
Bean	An object that Spring makes and controls.
Layer	A group of classes with the same job.
Controller	A class that gets the request from the browser.
DTO	A simple object that carries data between the layers.
Entity	A class that is connected to a database table.
Lote (batch)	A group of tickets for one game, with a price and a sale period.
Architecture pattern	A model for the structure of the complete system.
Design pattern	A model of a solution for a problem with classes and objects.
Repository	An interface that reads and writes data in the database.
Service	A class that applies the business rules.

Sistema de Sócio-Torcedores

Versão em Leitura Fácil

Este texto explica o nosso projeto de um jeito simples.

Ele é para quem não conhece programação.

Outras versões deste documento:

- Documentação completa em português
 - Documentation in English
-

1. O que é este projeto?

Este projeto é um **sistema de computador**.

Um sistema é um programa que ajuda a fazer uma tarefa.

O nosso sistema ajuda um **clube de futebol**.

O clube tem torcedores que pagam uma mensalidade.

Esses torcedores se chamam **sócio-torcedores**.

O sistema ajuda o clube a cuidar deles.

2. Quem fez o projeto?

Quatro estudantes de Ciência da Computação fizeram o projeto.

Eles estudam no **IFSul**.

O projeto é um trabalho da disciplina **LPOO**.

LPOO quer dizer Linguagem de Programação Orientada a Objetos.

Os estudantes são:

- Allan Theodoro
- Bernardo Zavistanovicz Deters
- Olavo Appel Neto
- Thomas Cansian dos Santos

3. O que o sistema faz hoje?

O sistema já faz estas coisas:

- **Cadastrar uma pessoa.** A pessoa escreve o nome de usuário, o e-mail, a senha e o CPF.
- **Entrar no sistema.** A pessoa usa o nome de usuário e a senha. Isso se chama **login**.
- **Sair do sistema.** Isso se chama **logout**.
- **Mostrar os jogos do ano.** A pessoa escolhe um mês. O sistema mostra os jogos desse mês.
- **Mostrar o próximo jogo.**
- **Mostrar os lotes de ingressos de um jogo.**

Um **lote** é um grupo de ingressos.

Cada lote tem um preço e um período de venda.

O sistema avisa se o lote está aberto ou fechado para venda.

4. O que o sistema ainda não faz?

Algumas coisas ainda não funcionam:

- Comprar o ingresso.
- Virar sócio do clube.
- Dar desconto para cada tipo de sócio.
- Dar brindes aos sócios.

Os estudantes querem fazer essas coisas depois.

5. Quem pode usar cada tela?

Uma **tela** é uma página que a pessoa vê no navegador.

Tela	Quem pode ver
Início	Todas as pessoas
Entrar	Todas as pessoas
Cadastro	Todas as pessoas
Jogos	Quem fez login
Ingressos	Quem fez login
Perfil	Quem fez login
Administração	Só o administrador

O **administrador** é a pessoa que cuida do sistema.

6. Como o sistema é feito?

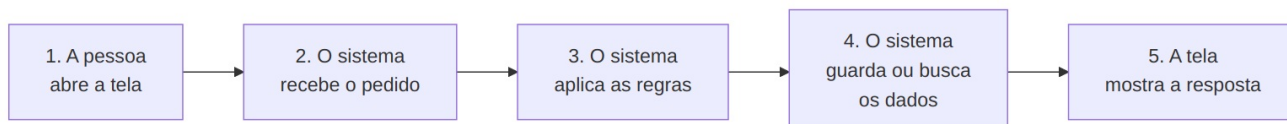
O sistema tem **partes**.

Cada parte tem um trabalho.

Essas partes se chamam **camadas**.

Camada	Trabalho dela
Telas	Mostrar as páginas para a pessoa
Serviços	Seguir as regras do clube
Dados	Guardar e buscar as informações

Veja como as camadas trabalham juntas:



Esta forma de organizar o sistema se chama **arquitetura em camadas**.

Outra forma usada é o **MVC**.

MVC divide o sistema em três partes: **Modelo**, **Visão** e **Controle**.

- O **Modelo** guarda as informações.
- A **Visão** é o que a pessoa vê.
- O **Controle** recebe o pedido da pessoa.

7. O que são padrões de projeto?

Um **padrão de projeto** é uma boa solução que muita gente já usou.

É como uma receita de bolo.

A receita ajuda a resolver um problema comum.

Nós usamos cinco padrões:

1. **Cadeia de Responsabilidade.** O sistema confere o lote em passos. Cada passo é uma regra. Hoje temos uma regra: o lote só vende no período certo.
2. **Método Modelo.** Todas as regras seguem o mesmo caminho.
3. **Fábrica.** Uma parte do sistema só faz objetos. Por exemplo, ela monta o cadastro de uma pessoa e esconde a senha.
4. **Construtor.** O sistema monta objetos grandes com calma, um campo de cada vez.
5. **Repositório.** Uma parte do sistema só conversa com o banco de dados.

O **banco de dados** é o lugar onde o sistema guarda as informações.

8. Como a pessoa usa o sistema?

Estes são os passos de uma pessoa nova:

1. A pessoa abre a página inicial.
2. A pessoa clica em **Entrar**.
3. A pessoa clica em **Cadastre-se**.
4. A pessoa preenche o cadastro.
5. A pessoa faz o login.
6. A pessoa clica em **Jogos**.
7. A pessoa vê o calendário.
8. A pessoa clica no botão do próximo jogo.
9. A pessoa vê os lotes de ingressos.

9. O que tem nas telas?

As telas têm estas partes:

- **Botões.** A pessoa clica neles.
- **Campos.** A pessoa escreve neles.
- **Links.** A pessoa clica e vai para outra tela.
- **Cartões.** Eles mostram um jogo ou um lote.
- **Menu.** Ele abre quando a pessoa clica na foto do perfil.
- **Escudos.** São as imagens dos times.

Quando a pessoa clica, o sistema responde.

Isso se chama **evento**.

Exemplo: a pessoa clica no mês de **outubro**.

O sistema mostra só os jogos de outubro.

10. O sistema foi testado?

Sim. Os estudantes fizeram **14 testes** à mão.

Veja o resultado:

Resultado	Quantidade
Deu certo	4
Deu erro	7
Falta decidir a regra	2
Só observamos	1

Os estudantes já sabem onde estão os erros.

Eles vão corrigir os erros.

11. O que vem depois?

Os próximos passos são:

1. Mostrar mensagens de erro claras no cadastro.
2. Evitar cadastros com dados errados.
3. Permitir a compra de ingressos.
4. Criar mais regras para os lotes.
5. Fazer mais testes.

12. Palavras difíceis

Palavra	O que quer dizer
Sistema	Programa que ajuda a fazer uma tarefa
Sócio-torcedor	Torcedor que paga mensalidade ao clube
Login	Entrar no sistema com nome e senha
Camada	Parte do sistema com um trabalho só
Padrão de projeto	Boa solução para um problema comum
Banco de dados	Lugar onde o sistema guarda informações
Evento	Resposta do sistema a um clique
Lote	Grupo de ingressos com preço e período de venda